

PROJECT OVERVIEW & KICKOFF REPORT

Fraud Detection System

Medical Insurance Claim Fraud Detection Using AI & Code Analysis

Cornell University Capstone | Shravani Poman | April 11, 2026

For: Project Teammates + Executive Sponsor Team

1. The WHAT — What Are We Building?

We are building an AI-powered fraud detection system for medical insurance claims. The system takes a single medical claim form — exactly as it arrives in a real insurance company's inbox — and automatically determines whether it is fraudulent, what type of fraud it contains, and why it was flagged.

In One Sentence

We built a system that reads a medical insurance claim PDF, checks the procedure codes against the diagnosis codes and prices, and flags fraud — without needing any other document or patient history to compare against.

What the System Does Step by Step

1. **Receives** a CMS-1500 medical claim form as a PDF — the standard form used by doctors and clinics across the entire United States.
2. **Reads** every field on the form using OCR (Optical Character Recognition) — extracting the procedure codes, diagnosis codes, amounts charged, and any special billing modifiers.
3. **Runs rule-based checks** for fraud patterns that follow fixed, provable logic — like checking if the same procedure was billed twice, or if a screening test was submitted without the required diagnosis code.
4. **Asks Claude AI** to reason about whether the combination of codes makes medical sense — for example, whether billing an oncology genomic test makes sense when the patient's diagnosis is back pain.
5. **Returns a verdict:** Fraud or Legitimate, the specific fraud type, and a plain-English explanation that an insurance reviewer can act on immediately.

2. The WHY — Why Does This Problem Need Solving?

The Scale of the Problem

Medical billing fraud is one of the largest financial crimes in the United States. The FBI estimates that healthcare fraud costs the country between \$68 billion and \$230 billion every year — roughly 3 to 10 cents of every dollar spent on healthcare. Every fraudulent claim that gets paid is money taken directly from insurance pools that fund legitimate patient care.

Why Existing Systems Are Failing

Most insurance companies today rely on two approaches to catch fraud, and both have critical gaps:

- **Manual review by humans:** A human reviewer looks at flagged claims. This is slow, expensive, inconsistent, and only catches a tiny fraction of the fraud that occurs. Reviewers get tired, make mistakes, and cannot scale to millions of claims.
- **Historical pattern analysis:** Systems that look for suspicious patterns in a provider's claim history over time — like a doctor who bills unusually high amounts. But this requires months of data to detect, meaning fraud goes undetected for a long time, and it completely misses first-time fraudsters.

The Critical Gap We Are Filling

Neither approach can answer the most fundamental question: Is THIS single claim, RIGHT NOW, internally fraudulent? That is exactly what our system does — it evaluates every claim in isolation, the moment it arrives, using only the information in that one document.

Why This Is Hard — And Why AI Is the Right Tool

The challenge is that fraud is not always obvious. A fraudulent claim often looks perfectly normal on the surface — the CPT codes are real, the ICD-10 codes are real, the amounts are within range. The fraud is hidden in the relationship between them. Consider these examples:

- **A claim for CPT 0048U** (an advanced oncology DNA sequencing test costing \$5,000+) with a diagnosis of J06.9 (common cold). Both codes are real and valid. But no doctor would order a cancer genomic test for a cold. A rule-based system looking only at individual codes would miss this. A human might catch it after spending minutes reviewing it. Our LLM catches it in under a second.
- **A claim for CPT 80061** (lipid panel) with modifier -59 applied alongside CPT 82465 (cholesterol test). Both codes look fine individually. But the lipid panel already includes the cholesterol test — billing them separately with modifier -59 is unbundling fraud. Our rule-based check catches this instantly.

3. The 9 Fraud Types We Detect — In Plain English

Here is every fraud type the system detects, what it means in simple terms, and how the system catches it:

#	Fraud Type	What It Means in Plain English	Detection	Risk
1	Upcoding	A doctor bills for a more expensive service than what actually happened. Example: patient had a quick 5-minute check-up but the claim says it was a 45-minute complex consultation.	LLM	● High
2	Unbundling	One procedure is split into multiple separate billing codes to charge more. Example: a lab panel that should be billed as one item is broken into 5 individual tests.	Rule	● High
3	Code Padding	Extra unrelated codes are added to a legitimate claim to inflate the total. Example: a routine office visit has expensive cancer genomic tests added that were never ordered.	LLM	● High
4	Phantom Billing	Billing for a procedure that was never performed at all. Example: a patient came in for a cold and the claim includes a heart catheterization that never happened.	LLM	● High

5	Diagnosis Mismatch	The procedure code and the diagnosis code have no logical medical relationship. Example: billing an oncology gene test but the diagnosis says the patient had back pain.	LLM	● Medium
6	Code Substitution	A non-covered or cosmetic procedure is swapped for a code that insurance will pay for. Example: a cosmetic consultation is billed as a medically necessary office visit.	LLM	● Medium
7	Modifier Abuse (-59)	A special billing flag called modifier -59 is misused to make two procedures that are supposed to be billed together appear to be separate, bypassing insurance rules.	Rule	● Medium
8	Duplicate Billing	The exact same procedure code is submitted more than once for the same patient visit. Example: a blood test appears twice on the same claim.	Rule	● Low
9	Screening Code Abuse	A screening test is billed without the required medical justification code. Example: billing a colorectal cancer screening but the diagnosis code is for hypertension, not the required screening code.	Rule	● Low

Rule vs. LLM — Why We Use Both

Rule-based detection is instant, 100% accurate, and costs nothing to run. We use it for fraud types with clear, fixed patterns like duplicate billing.

LLM reasoning is needed for fraud types that require medical judgment — understanding that a cardiac catheterization billed during a routine wellness visit is clinically impossible. Rules cannot capture this nuance. Claude AI can.

4. The Data — Where It Comes From and Why We Built It

Why We Generated Synthetic Data

Real medical billing fraud data is almost impossible to obtain. It is protected by HIPAA privacy laws, held by insurance companies who treat it as proprietary, and comes with no clean fraud labels — in the real world, nobody writes 'this is fraudulent' on a claim. Without labeled data, you cannot train or evaluate a supervised machine learning system.

So we built our own. We used the 2026 DHS Code List Addendum — the official government list of 1,500+ real CPT/HCPCS procedure codes published by the Centers for Medicare & Medicaid Services — as the source of truth for which procedure codes are valid. We paired these with real ICD-10 diagnosis codes and deliberately constructed fraudulent combinations that mirror the patterns described in fraud prosecution cases and insurance industry reports.

What the 2026 DHS Code List Is

The DHS Code List Addendum (formally titled 'List of CPT/HCPCS Codes Used to Define Certain Designated Health Service Categories Under Section 1877 of the Social Security Act') is a government document that lists every valid medical procedure code that can be billed under Medicare and Medicaid. Using this list means every CPT code in our dataset is a real, currently valid billing code — not a made-up one. This makes our synthetic claims realistic enough to train a model that will generalize to real-world claims.

Why CMS-1500 Specifically

The CMS-1500 is the universal standard claim form used by physicians, specialists, clinics, and outpatient facilities across the entire United States. It is mandated by CMS (the Centers for Medicare and Medicaid Services) and accepted by virtually every insurance payer in the country. Choosing CMS-1500 means our system covers the widest possible range of real-world claims — office visits, lab tests, imaging, cardiac procedures, specialist consultations, and more.

The form has a specific, standardized layout: patient information in the top section, insurance information in the middle, and up to 6 procedure lines at the bottom — each with a CPT code, diagnosis pointer, modifier, and charge. This standardization is what makes automated OCR extraction reliable. Every CMS-1500 has the same fields in the same places.

Dataset Composition — 1,300 Claims

Fraud Type	Count	Why This Many
Legitimate (no fraud)	400	Baseline — model must learn what a normal claim looks like. More clean claims = better at avoiding false alarms.
Upcoding	120	Most common fraud type in real insurance data — given highest representation.
Diagnosis Mismatch	120	Second most common — needs strong signal for the LLM to learn from.
Unbundling	100	Very common in lab billing — needs enough examples for rule to be robust.
Code Padding	100	Common in multi-code claims — 100 gives good coverage of padding patterns.
Phantom Billing	100	High-value fraud — 100 examples cover the range of impossible CPT-ICD combinations.
Code Substitution	100	Subtle fraud — needs enough examples for LLM to learn the substitution pattern.
Modifier Abuse (-59)	100	Rule-based but needs enough variety across bundled code pairs.
Duplicate Billing	80	Simplest fraud — fewer needed since detection rule is straightforward.
Screening Code Abuse	80	Simpler pattern — fewer examples sufficient for rule accuracy.
TOTAL	1,300	Enough data to train, validate, and test the model rigorously.

5. What Was Built Today — Step by Step

Step 1 — Problem Redefinition

We started the day with a fundamental pivot. The original project was built around comparing a claim against an original document. The sponsor clarified that in the real world, you only ever receive one document — the claim itself. There is no original to compare against.

This changed everything. The entire approach was redesigned around single-document fraud detection — finding fraud signals within one claim using only the codes, diagnoses, and amounts present in that document, cross-referenced against medical knowledge about what combinations are clinically valid.

Step 2 — Fraud Taxonomy Design

We defined the 9 fraud types the system will detect, drawing on the sponsor's input, published insurance fraud research, and the document your sponsor provided (which itself described 9 fraud categories including upcoding, unbundling, code padding, phantom billing, diagnosis mismatch, code substitution, modifier abuse, duplicate billing, and screening code abuse).

For each fraud type, we determined: what the fraud looks like in terms of CPT and ICD-10 codes, whether it can be caught by a deterministic rule or requires LLM reasoning, how common it is in the real world, and how to represent it realistically in synthetic data.

Step 3 — Form Selection

We evaluated three standard medical billing forms: CMS-1500 (physician/outpatient), UB-04 (hospital/inpatient), and the ADA Dental Claim Form. We chose CMS-1500 because it covers the broadest range of claims, has the simplest and most standardized structure for OCR extraction, and aligns with the use case the sponsor described. The system architecture is designed so that UB-04 support can be added later by building a new field extractor while reusing the entire fraud detection engine unchanged.

Step 4 — Dataset Generator Built and Run

We wrote `generate_claims.py` — a Python script that generates synthetic CMS-1500 claims using real CPT codes from the 2026 DHS Code List. The script was designed with the following principles:

- **Real codes only:** Every CPT and ICD-10 code used is a real, valid medical billing code. No made-up codes.
- **Clinically grounded:** Legitimate claims pair codes that make medical sense — a lipid panel with a cardiovascular screening diagnosis, an ECG with a cardiac condition. Fraud claims deliberately break these relationships in specific, documented ways.
- **Realistic prices:** Every procedure has a price range based on real Medicare reimbursement rates. A blood test costs \$20–60, not \$5,000.
- **Complete ground truth:** Every claim has a `fraud_type` label and a `fraud_explanation` field that explains in plain English exactly what the fraud is and why.
- **Reproducible:** Fixed random seed (42) means the dataset is identical every time the script runs.

The script was executed and produced 1,300 claims in under 5 seconds, saved as JSON and CSV.

Step 5 — Files Produced

File	Format	What It Contains & Why It Exists
<code>generate_claims.py</code>	Python script	The code that generates all 1,300 synthetic claims. Re-run anytime to regenerate the dataset with a different random seed. Lives in <code>src/</code> .
<code>claims.json</code>	JSON	All 1,300 claims in full detail — every field, every label, every explanation. This is the ground truth the model trains on. Lives in <code>data/raw_claims/</code> .

claims_summary.csv	CSV / Excel	Human-readable summary of all claims — open in Excel to browse and inspect the dataset. Same data as JSON but easier for humans to read. Lives in data/raw_claims/.
--------------------	-------------	---

6. The Full Pipeline — How It Will All Connect

Here is the complete end-to-end pipeline we are building, with every step explained:

Step	Stage	What Happens	Why It's Needed
1	PDF Intake	The system receives a CMS-1500 medical claim form as a PDF file — exactly as it would arrive in a real insurance workflow.	<i>This is the real-world entry point. Claims arrive as PDFs, not spreadsheets, so the system must handle raw documents.</i>
2	OCR Extraction	Optical Character Recognition (pytesseract) reads the PDF and pulls out all the text — procedure codes, diagnosis codes, patient info, amounts.	<i>A computer can't analyze a PDF like a human reads a page. OCR converts the image into structured, machine-readable data.</i>
3	Field Parsing	The extracted text is organized into a standard structure: which CPT codes were billed, which ICD-10 diagnoses, what amounts, any modifiers.	<i>Raw OCR text is unstructured. This step maps it to the exact fields the fraud detection engine expects.</i>
4	Rule-Based Checks	Four fraud types (duplicate billing, unbundling, modifier abuse, screening code abuse) are checked using deterministic Python rules — no AI needed.	<i>These fraud types follow fixed, provable patterns. A rule that says 'if the same code appears twice, flag it' is 100% accurate and instant.</i>
5	LLM Reasoning	Claude AI evaluates the remaining five fraud types (upcoding, code padding, phantom billing, diagnosis mismatch, code substitution) by reasoning about whether the codes make clinical sense together.	<i>These fraud types require medical judgment — understanding whether a cardiac test makes sense for a back pain patient. Only an LLM can reason about this contextually.</i>
6	Output	The system returns: Fraud / Not Fraud, the specific fraud type detected, and a plain-English explanation of why it was flagged.	<i>The output needs to be actionable. An insurance reviewer needs to know not just 'fraud' but exactly what the problem is and why.</i>

7. The Build Roadmap — 40 Hours to a Working System

We have approximately 40 hours of build time remaining. Here is exactly what gets built when:

Hours	Step	What Gets Built	Status
0–2	Dataset Generation	1,300 synthetic CMS-1500 claims generated as JSON — 400 legitimate + 900 fraud across all 9 types using real 2026 DHS CPT codes	✅ DONE
2–6	PDF Renderer	Each JSON claim rendered as a realistic CMS-1500 PDF using reportlab — 1,300 PDFs produced	🕒 Next
6–10	OCR	pytesseract reads each PDF and extracts structured fields: CPT codes, ICD-10,	Upcoming

	Pipeline	amounts, modifiers, patient info	
10–14	Rule-Based Checks	Python rules detect: duplicate billing, unbundling, modifier abuse (-59), screening code abuse — fast and deterministic	Upcoming
14–22	LLM Reasoning Layer	Claude API evaluates: upcoding, code padding, phantom billing, diagnosis mismatch, code substitution — one prompt per claim	Upcoming
22–26	Full Pipeline	Connect all steps end to end — PDF in, fraud flag out — run on all 1,300 claims	Upcoming
26–32	Evaluation	Precision, recall, F1 per fraud type. Target: binary fraud F1 ≥ 0.85 on held-out test set	Upcoming
32–40	Streamlit Demo App	Upload any CMS-1500 PDF → get fraud score, fraud type, and plain-English explanation in seconds	Upcoming

8. For the Executive Sponsor Team — The Business Case

What You Are Getting

A working AI system that takes any CMS-1500 medical claim PDF as input and returns a fraud verdict with an explanation in seconds. The system covers 9 distinct fraud types — from simple duplicate billing to complex diagnosis-procedure mismatches that currently require expensive manual review to detect.

Why This Approach Is Different From What Exists

- **Existing tools analyze provider patterns over time.** Our system evaluates each individual claim the moment it arrives — catching fraud before it is paid, not months later.
- **Existing tools require historical data.** Our system needs only the claim itself — making it equally effective on first-time fraudsters and established providers.
- **Existing tools are black boxes.** Our system explains every decision in plain English — giving reviewers the specific reason a claim was flagged so they can act immediately.

What Makes This Technically Rigorous

- Uses real CPT/HCPCS codes from the official 2026 CMS code list — not approximations.
- The LLM reasoning layer uses Claude, which has been trained on medical literature and understands clinical relationships between diagnoses and procedures.
- The system is evaluated on a held-out test set it has never seen, using precision, recall, and F1-score — the standard metrics for fraud detection systems.
- The architecture is modular — rule-based checks and LLM reasoning are separate components that can be updated independently as fraud patterns evolve.

Current Status: Project foundation complete. 1,300 labeled synthetic claims generated using real 2026 DHS CPT codes. Next step: render each claim as a CMS-1500 PDF and begin the OCR extraction pipeline.